Rapport du projet

Khadidiatou Coura Sabaly

April 2026

Table des matières

A Introduction **1**

B Manuel d'utilisation **2**

C Choix de structures de données **2**
C.1 Représentation des types 2
C.2 Représentation des valeurs 2
C.3 Représentation d'une table 2
C.4 Représentation des dépendances fonctionnelles 2

D Organisation du programme **2**

E Problèmes techniques rencontrés **2**
E.1 Vérification des types des lignes 2
E.2 Calcul des dépendances fonctionnelles 3
E.3 Recherche des clés 3
E.4 Détection des formes normales 3

F Solutions apportées **3**
F.1 Utilisation de fonctions récursives 3
F.2 Factorisation dans des fonctions utilitaires 3
F.3 Découpage en modules 3
F.4 Réduction des dépendances à une seule colonne 3
F.5 Vérification progressive 3

G Jeux de tests **4**

H Limites du programme **4**

I Conclusion **4**

A Introduction

Ce projet a pour objectif d'implémenter, en OCaml, un petit système d'analyse de tables relationnelles. Le programme manipule des schémas de tables, des lignes de données, des dépendances fonctionnelles, des clés candidates et permet d'estimer le niveau de normalisation d'une table.

B Manuel d'utilisation

Pour compiler le projet, il suffit d'exécuter la commande `make` dans le répertoire du projet. L'exécutable obtenu peut ensuite être lancé avec `./test`.

Le programme est organisé en plusieurs modules : `types`, `utils`, `table`, `deps` et `normalization`. Le fichier `projet.ml` contient un exemple d'utilisation. Les fichiers `test_operation.ml` et `test_dep.ml` permettent de vérifier les principales fonctionnalités du projet. Pour nettoyer les fichiers générés taper `make clean` et `make htmlclean`.

C Choix de structures de données

Le programme repose sur quelques types simples.

C.1 Représentation des types

Les valeurs de base ont été limitées à deux types primitifs : les entiers et les textes.

C.2 Représentation des valeurs

Une valeur de table peut être un entier, une chaîne de caractères, ou la valeur nulle. La présence de `VNull` permet de représenter des attributs optionnels.

C.3 Représentation d'une table

Une table est représentée par un schéma et une liste de lignes. Le schéma associe à chaque nom de colonne un type et une information de nullabilité.

C.4 Représentation des dépendances fonctionnelles

Une dépendance fonctionnelle est modélisée par un couple `(lhs, rhs)` où `lhs` et `rhs` sont des listes de noms de colonnes.

D Organisation du programme

Pour améliorer la lisibilité, le projet a été découpé en plusieurs modules.

— `types.ml` contient uniquement les définitions de types.
— `utils.ml` regroupe les fonctions utilitaires communes.
— `table.ml` contient les opérations de base sur les tables.
— `deps.ml` traite les dépendances fonctionnelles et les clés.
— `normalization.ml` calcule le niveau de normalisation.
— `projet.ml` sert de fichier de test.
— `Makefile` pour la compilation.

E Problèmes techniques rencontrés

E.1 Vérification des types des lignes

L'un des premiers problèmes a été de vérifier qu'une ligne respecte bien le schéma de la table. Il fallait contrôler à la fois la longueur de la ligne et la compatibilité entre chaque valeur et le type de la colonne correspondante.

E.2 Calcul des dépendances fonctionnelles

Le calcul des dépendances fonctionnelles à partir des données présentes dans la table est coûteux, car il faut comparer plusieurs combinaisons de colonnes et plusieurs paires de lignes. Le nombre de combinaisons possibles augmente rapidement avec le nombre d'attributs.

E.3 Recherche des clés

La recherche de clés candidates pose un problème similaire, car elle nécessite d'explorer plusieurs sous-ensembles de colonnes.

E.4 Détection des formes normales

L'évaluation de la 2NF, 3NF et BCNF dépend des dépendances élémentaires et des clés trouvées. Il faut donc enchaîner correctement plusieurs traitements intermédiaires avant de pouvoir conclure.

F Solutions apportées

F.1 Utilisation de fonctions récursives

Le programme utilise des fonctions récursives pour parcourir les listes de lignes, de colonnes et de dépendances.

F.2 Factorisation dans des fonctions utilitaires

Les fonctions fréquemment réutilisées, comme l'extraction des noms de colonnes ou la comparaison de listes de valeurs, ont été regroupées dans un module utilitaire.

F.3 Découpage en modules

La séparation en plusieurs fichiers permet de mieux structurer le programme. Chaque module expose uniquement les fonctions utiles, ce qui limite les dépendances inutiles entre parties du code.

F.4 Réduction des dépendances à une seule colonne

Pour analyser plus facilement les dépendances fonctionnelles, les dépendances à partie droite multiple ont été transformées en dépendances mono-colonnes (RHS contenant une seule colonne). Cela simplifie la recherche de dépendances élémentaires et la vérification des formes normales.

F.5 Vérification progressive

Le calcul du niveau de normalisation est effectué par étapes :
— calcul des dépendances observées ;
— extraction des dépendances élémentaires ;
— recherche des clés ;
— tests successifs de la BCNF, de la 3NF puis de la 2NF.

G Jeux de tests

Deux fichiers de test ont été utilisés pour valider le programme.

Le fichier `test_operation.ml` vérifie les opérations de base sur les tables : `insert`, `projection`, `restrict` et `prod`. Ces tests servent à contrôler que les transformations de tables respectent bien le schéma et les lignes attendues.

Le fichier `test_dep.ml` vérifie le calcul des dépendances fonctionnelles, des dépendances élémentaires et du niveau de normalisation. La table utilisée a été construite manuellement pour produire des dépendances observables et tester le comportement du programme sur un cas non trivial.

H Limites du programme

Le programme présente plusieurs limites.

— La recherche de dépendances et de clés repose sur l'exploration de nombreuses combinaisons, ce qui peut devenir coûteux lorsque le nombre de colonnes augmente.

— Le programme ne gère que deux types primitifs de base, les entiers et les textes.

— Les exceptions sont utilisées pour signaler certaines erreurs d'utilisation, mais un traitement plus fin des erreurs pourrait être envisagé.

I Conclusion

Ce projet m'a permis de mettre en pratique plusieurs notions importantes de modélisation relationnelle et de programmation en OCaml. Le travail de structuration en modules a amélioré la lisibilité du code et facilité son évolution.

Les choix effectués visent avant tout la clarté, la simplicité et la correction fonctionnelle. Même si le programme reste limité en performance et en généralité, il répond aux objectifs demandés et fournit une base propre pour des améliorations futures.